

Automatisation et fiabilisation des flux de données dans une agence de gestion de patrimoine

Data Analyst & Développeur BI

SARL Renaud Besson – Le Conservateur

Direction Régionale Ouest – Niort / La Roche-sur-Yon

IUT de Niort – Promotion 2025-2026
Tuteur académique : M. Mesmoudi
Maître de stage : M. Renaud Besson

Remerciements

En préambule à ce rapport de stage, je tenais tout d'abord à remercier toutes les personnes qui ont contribué au bon déroulement de celui-ci en m'accueillant au sein de leur structure du 6 avril au 12 juin 2026.

Un grand merci avant tout à Monsieur Renaud Besson, mon maître de stage et directeur de la région Ouest pour Le Conservateur qui, dès le début, m'a fait pleinement confiance et m'a laissé une réelle liberté sur le plan technique ce qui est peu courant pour un stage étudiant. Ses conseils, sa façon de concevoir le métier de la gestion de patrimoine et son niveau d'exigence m'ont vraiment poussé à donner le meilleur de moi-même.

Je remercie également tous les collaborateurs de l'agence et particulièrement Laura qui a fait preuve d'une grande disponibilité à mon égard. Nos échanges m'ont beaucoup aidé à comprendre le mode de fonctionnement de l'agence pour mieux évaluer les besoins réels des potentiels utilisateurs afin de leur proposer, par la suite, des solutions les plus pertinentes possibles.

Enfin, merci à l'équipe pédagogique de l'IUT de Niort, et plus particulièrement à M. Mesmoudi, mon tuteur académique, pour son suivi tout au long de ce stage. Les bases acquises pendant ma formation ont été indispensables pour mener ce projet à bien.

Sommaire

Introduction

1. Le Conservateur et l'agence Renaud Besson

1.1. Présentation du groupe Le Conservateur

1.2. La SARL Renaud Besson

1.3. Mon intégration au sein de l'agence

2. État des lieux : ce qui ne fonctionnait pas

2.1. Comment se passait la préparation des rendez-vous

2.2. Les conséquences concrètes pour l'équipe

3. L'application Python que j'ai développée (Missions 1 & 2)

3.1. Les choix techniques

3.2. Extraction des données depuis les PDF et les CSV

3.3. Génération automatique des fichiers finaux

4. La refonte du CRM Excel (Mission 3)

4.1. Centralisation de la base de données

4.2. Développement des macros VBA

4.3. Confidentialité et RGPD

5. Bilan : résultats, retours et compétences acquises

5.1. Ce que ça a changé concrètement

5.2. Mise en place, manuel et retours de l'équipe

5.3. Ce que j'ai appris

Conclusion générale

Résumé (FR / EN)

Annexes

Introduction générale

Aujourd'hui, dans le domaine de la gestion de patrimoine, les clients ne se satisfont plus d'un conseiller qui se contente de leur vendre des produits financiers. Ils attendent quelqu'un capable d'analyser leur situation fiscale, de faire des projections sur le long terme et de leur présenter tout cela de manière claire et professionnelle.

Mais pour arriver à ce niveau de qualité, les conseillers ont besoin que leurs données soient fiables, accessibles et bien organisées. Et c'est là le point faible que connaissent beaucoup d'agences. Les informations sont éparpillées partout : dans des PDF, sur des plateformes internes fermées, dans des fichiers Excel faits maison. Résultat : beaucoup de temps perdu, des risques d'erreurs et une charge administrative qui finit par peser lourd sur toute l'équipe.

C'est exactement ce problème que j'ai été chargé de résoudre lors de mon stage au sein de la SARL Renaud Besson, une agence affiliée au groupe Le Conservateur, spécialisé dans l'ingénierie patrimoniale haut de gamme. Sous la responsabilité directe de Monsieur Renaud Besson, directeur de la région Ouest, ma mission était claire : analyser comment l'information circule dans l'agence, trouver les blocages, et construire des outils logiciels sur mesure pour automatiser ce qui pouvait l'être.

Avant mon arrivée, préparer un dossier client prenait entre deux et trois heures. Le conseiller devait éplucher des PDF à la main, recopier des données dans Excel depuis des captures d'écran, puis tout reformater dans PowerPoint. Un travail long, répétitif et source d'erreurs dans un secteur où la moindre faute sur un chiffre peut avoir de vraies conséquences.

Mon travail a donc consisté à construire de A à Z un écosystème d'outils automatisés : une application web en Python capable de lire les PDF et de générer les fichiers finaux en quelques secondes, un déploiement sécurisé via Docker pour que ça fonctionne partout sans problème, et la refonte d'un outil CRM sous Excel qui était à l'abandon.

Ce rapport explicite donc la démarche que j'ai choisi de suivre : d'abord la présentation de l'entreprise et son contexte, ensuite le diagnostic de l'existant, puis le détail technique de ce que j'ai conçu, et enfin le bilan de ce que ces semaines de stage ont apporté à l'agence et à moi personnellement.

1. Le Conservateur et l'agence Renaud Besson

Avant de parler code et automatisation, il était important pour moi de bien cibler l'environnement dans lequel j'allais travailler. Un développeur qui ne comprend pas le métier de ses utilisateurs débouche sur la conception d'outils qui ne servent à rien. Alors dès les premiers jours, j'ai pris le temps de découvrir, apprendre et comprendre.

1.1. Présentation du groupe Le Conservateur

1.1.1. Un groupe indépendant depuis 1844

Le Conservateur existe depuis 1844 soit depuis plus de 170 ans ! Ce qui est particulier avec ce groupe, c'est qu'il ne ressemble pas aux grandes banques ou compagnies d'assurance classiques. C'est un groupe mutualiste, ce qui veut dire qu'il n'a pas d'actionnaires à satisfaire. Ses clients en sont les membres. Cette structure lui permet de se concentrer uniquement sur l'intérêt de ses sociétaires sans se préoccuper de faire monter un cours en bourse ou de verser des dividendes chaque trimestre.

Avec plusieurs milliards d'euros gérés, Le Conservateur cible une clientèle aisée : des cadres supérieurs, des médecins, des chefs d'entreprise, des professions libérales. Des personnes qui ont un patrimoine à gérer et qui cherchent des solutions solides sur le long terme.

1.1.2. La Tontine, le produit phare

Le Conservateur propose plusieurs produits : assurances-vie, plans d'épargne retraite, contrats de prévoyance... Mais ce qui le distingue vraiment de ses concurrents, c'est la Tontine. C'est un produit sur lequel il a quasiment le monopole en France.

Comment ça fonctionne ? Un groupe d'épargnants mettent leur argent en commun dans une « association tontinière » pour une durée fixe, entre 10 et 25 ans. Cet argent est investi sur les marchés financiers (obligations, immobilier, actions). À la fin, tout le capital généré est partagé uniquement entre les membres encore en vie à cette date.

Pour un développeur, ce produit est un vrai casse-tête mathématique. Les projections financières ne se calculent pas avec une simple formule. Il faut tenir compte de l'âge du client à l'ouverture du contrat (qui détermine son espérance de vie et donc sa part de la

mise), de la durée d'investissement, des options de prévoyance choisies (couverture décès, revalorisation ou non) ... Bref, des dizaines de variables interdépendantes. C'est justement pour cela qu'automatiser ces calculs prenaient tout son sens.

1.1.3. Le profil des clients et les exigences qui vont avec

Les clients du Conservateur ne sont pas du genre à se contenter d'un tableau Excel bâclé. Ce sont des professionnels qui comprennent les chiffres et qui attendent des documents parfaits aussi bien sur le fond que sur la forme. Un rapport mal présenté ou avec une erreur dans une projection à 20 ans peut entamer sérieusement la confiance dans le gestionnaire et/ou l'agence.

Le conseiller a donc un vrai devoir de conseil légal et moral. Et pour remplir ce rôle correctement, il a besoin d'outils fiables.

1.2. La SARL Renaud Besson

1.2.1. Une agence régionale sur deux sites

Le Conservateur ne fonctionne pas avec des agences ouvertes au public comme une banque classique. Il s'appuie sur un réseau d'agents mandatés, des entrepreneurs indépendants qui gèrent leur propre cabinet tout en commercialisant les produits du groupe.

C'est dans ce cadre que s'inscrit la SARL Renaud Besson. Monsieur Besson est directeur de la région Ouest. Son équipe composée de sept collaborateurs commerciaux indépendant est répartie entre deux bureaux : un à Niort (la capitale des mutuelles) et un à La Roche-sur-Yon.

1.2.2. Une équipe toujours sur le terrain

Si vous passez dans ces bureaux en pleine journée, il y a de grandes chances que vous les trouviez vides. Et c'est normal. Les conseillers en gestion de patrimoine passent leur temps en rendez-vous chez les clients ou en déplacement entre deux rendez-vous. Ils ne sont devant leur ordinateur en agence que de façon ponctuelle.

Ce mode de fonctionnement très mobile a un impact direct sur la manière dont ils travaillent. Chaque heure passée à faire de la saisie administrative sur ordinateur, c'est une heure de moins pour voir des clients. D'où l'urgence de réduire au maximum le temps de back-office.

1.3. Mon intégration au sein de l'agence

1.3.1. Un rôle double : analyste et développeur

Mon poste n'était pas un simple stage d'assistance informatique. J'avais une vraie feuille de route : analyser le fonctionnement existant, trouver les problèmes et concevoir des solutions logicielles de bout en bout. Pour résumer, j'étais à la fois la personne qui comprend et modélise la donnée (Data Analyst) et celle qui construit les outils (développeur d'applications).

1.3.2. Une méthode de travail agile

Monsieur Besson a joué le rôle de Product Owner : c'est lui qui définissait les besoins, me transmettait les règles de calcul et me donnait son retour sur ce que je développais. De mon côté, j'avais toute latitude pour choisir les technologies utilisées.

On avait un point de synchronisation chaque lundi matin pour faire le bilan de la semaine et définir les priorités suivantes. Entre ces réunions, on communiquait surtout via Microsoft Teams, ce qui était pratique étant donné que l'équipe était rarement toute réunie au même endroit.

2. État des lieux : ce qui ne fonctionnait pas

Avant de commencer à coder quoi que ce soit, j'ai passé mes premières semaines à observer et comprendre comment les conseillers travaillaient. Et ce que j'ai vu m'a vraiment surpris. Pas parce que les gens ne sont pas compétents, bien au contraire. Mais parce que les outils dont ils disposent les obligent à perdre un temps considérable sur des tâches qui pourraient être automatisées.

2.1. Comment se passait la préparation des rendez-vous

2.1.1. Des données enfouies dans des PDF impossibles à exploiter directement

La première source d'informations pour préparer un dossier client, ce sont les documents contractuels envoyés par le siège : relevés annuels, contrats de Tontine, conditions particulières des assurances-vie. Ces fichiers peuvent faire plusieurs dizaines de pages avec beaucoup de textes juridiques et quelques données chiffrées importantes disséminées un peu partout.

Pour récupérer ces données (le montant investi, la date de souscription, les options choisies...), le conseiller doit les chercher à la main dans le document, les lire puis les recopier ailleurs. Un travail long et fastidieux surtout quand les collaborateurs le font sur plusieurs dossiers à la suite.

2.1.2. Une plateforme interne qui montrait tout... mais ne laissait rien exporter

La deuxième source de données est une plateforme interne appelée Qlik Sense (ou « Click » dans le vocabulaire de l'équipe). C'est un outil de visualisation de données qui permet de suivre les performances financières des portefeuilles clients : valeur liquidative des supports, évolution des unités de compte, scores ESG...

Le problème ? Cette plateforme est parfaite pour regarder les données à l'écran mais impossible à connecter aux outils bureautiques de l'agence. Pas d'API ouverte, pas de bouton d'export en CSV ou en Excel. Les données restent prisonnières du navigateur web.

2.2. Les conséquences concrètes pour l'équipe

2.2.1. Un processus manuel épuisant

Face à ces deux obstacles, les conseillers avaient trouvé une solution de fortune : les captures d'écran. Voilà comment ça se passait en pratique :

- Ils faisaient des captures d'écran des tableaux de bord Qlik Sense.
- Ils ouvraient Excel d'un côté et la capture d'écran de l'autre et ils recopiaient les données à la main, ligne par ligne.
- Une fois les données dans Excel, ils faisaient leurs calculs puis copiaient les résultats dans PowerPoint pour construire le support de présentation du rendez-vous.

Ce processus complet prenait entre 2h30 et 3 heures par dossier. Sur une semaine, cela représentait une part énorme du temps de travail gaspillée sur des tâches répétitives sans valeur ajoutée.

2.2.2. Des risques d'erreurs et des conséquences légales

Au-delà du temps perdu, il y avait un risque plus sérieux : celui de se tromper. Recopier des chiffres depuis une capture d'écran, c'est une tâche que l'humain fait naturellement moins bien que la machine. Un chiffre décalé d'une colonne, une décimale oubliée, une confusion entre deux lignes... et la projection financière à 20 ans devient fausse.

Dans un secteur comme la gestion de patrimoine, cela n'est pas anodin. L'Autorité des Marchés Financiers (AMF) surveille de près le respect du devoir de conseil. Présenter à un client une simulation basée sur une erreur de saisie, c'est un risque légal et commercial réel.

2.3. Ce qu'il fallait construire

À partir de ce constat, j'ai défini avec Monsieur Besson quatre axes de travail :

- Automatiser la lecture des PDF pour extraire les données importantes sans intervention humaine.
- Trouver une alternative à la capture d'écran pour récupérer les données de Qlik Sense.
- Centraliser tous les calculs financiers dans un script fiable plutôt que dans des tableurs fragiles.
- Générer automatiquement les fichiers finaux (Excel, PDF, PowerPoint) en un seul clic.

3. L'application Python que j'ai développée (Missions 1 & 2)

Une fois le diagnostic posé, j'ai commencé à construire la solution. Cette partie détaille les choix techniques que j'ai faits, les problèmes que j'ai rencontrés et comment je les ai résolus.

3.1. Les choix techniques

3.1.1. Python et Flask pour le cœur de l'application

Pour le développement, j'ai choisi Python. C'est le langage de référence pour tout ce qui touche à la manipulation de données : il existe des bibliothèques très matures pour lire des PDF, traiter des fichiers CSV, faire des calculs, générer des documents Excel ou PowerPoint...

Mais un script Python qui tourne dans une console, ça n'est pas utilisable par un conseiller commercial. Pour rendre l'outil accessible via un navigateur web, j'ai encapsulé mon code dans Flask, un micro-framework web léger. J'ai ainsi créé une petite application web avec une interface très simple : une zone de glisser-déposer (drag & drop) pour uploader les fichiers, et un bouton pour lancer le traitement. Pas besoin de formation technique pour s'en servir.

J'ai travaillé avec Visual Studio Code comme environnement de développement, avec GitHub Copilot intégré pour m'aider à prototyper plus vite.

3.1.2. Docker pour un déploiement sans surprise

L'agence a deux sites (Niort et La Roche-sur-Yon) et les conseillers ont des ordinateurs différents, avec des configurations différentes. Si j'avais installé mon application directement sur chaque machine, j'aurais eu des problèmes de compatibilité tôt ou tard : une version de Python différente ici, une bibliothèque manquante là, et l'outil plante.

Pour éviter ça, j'ai utilisé Docker. En termes simples, Docker permet d'empaqueter une application avec tout ce dont elle a besoin pour fonctionner (le système d'exploitation, les bibliothèques, les dépendances...) dans une sorte de « boîte » autonome appelée conteneur. Cette boîte tourne de la même façon sur n'importe quelle machine. Résultat :

l'application se comporte exactement pareil sur mon ordinateur de développement et sur les postes en production.

3.2. Extraction des données depuis les PDF et les CSV

3.2.1. Mission 1 : lire les PDF intelligemment

Le premier défi technique était d'extraire des données précises depuis des documents PDF contractuels qui pouvaient faire plusieurs dizaines de pages. Pour ça, j'ai utilisé la bibliothèque Python pdfplumber, qui permet de localiser et d'extraire du texte dans un PDF avec une très bonne précision.

Le problème, c'est que ces PDF ne sont pas structurés comme une base de données. Les données importantes sont noyées dans du texte juridique. J'ai donc développé des algorithmes basés sur des expressions régulières (regex) : ce sont des règles qui permettent de dire à l'ordinateur « cherche dans le texte quelque chose qui ressemble à un numéro de police sur 8 chiffres », ou « cherche un montant en euros avec des séparateurs de milliers ».

Concrètement, le programme parcourt le document de bout en bout, repère automatiquement les numéros de polices, les noms de produits (Tontine Prime Unique, Hélios Patrimoine...), les dates d'échéance, les montants investis... et les place dans une structure de données propre et utilisable. Ce qui prenait 20 à 30 minutes de lecture visuelle se fait maintenant en une fraction de seconde.

3.2.2. Mission 2 : l'échec de l'OCR et le pivot vers l'IA générative

Pour la deuxième source de données (les tableaux de bord Qlik Sense), mon idée de départ était d'utiliser un moteur OCR (reconnaissance optique de caractères). En clair, l'idée était de donner la capture d'écran à un algorithme qui lirait les chiffres dessus et les extrairait automatiquement. J'ai utilisé Tesseract OCR pour ça.

Mais dans la pratique, ça n'a pas fonctionné comme prévu. Les captures d'écran n'avaient pas toujours la même résolution selon l'ordinateur utilisé. Le zoom du navigateur variait. L'anticrénelage de Windows rendait certains pixels flous. Résultat : l'OCR se trompait régulièrement, confondant des « 6 » avec des « 8 », oubliant des signes négatifs, mélangeant des colonnes. Dans un domaine où la précision des chiffres est cruciale, un taux de fiabilité de 98% ne suffit pas.

J'ai donc abandonné cette approche et cherché une alternative. L'idée qui s'est imposée : puisqu'on ne peut pas exporter les données directement depuis Qlik Sense, pourquoi ne pas utiliser l'intelligence artificielle pour convertir les données affichées à l'écran en un fichier CSV propre ?

Voilà comment ça marche maintenant : j'ai créé un « prompt » standardisé (une instruction précise pour l'IA) que les conseillers peuvent copier-coller dans un outil IA grand public (comme ChatGPT ou Claude). L'IA lit les données de la capture d'écran ou du texte copié depuis Qlik Sense, et génère un fichier CSV parfaitement formaté. Ce fichier est ensuite glissé dans mon application, qui le lit avec la bibliothèque pandas sans aucune erreur. Taux d'erreur : zéro.

3.3. Génération automatique des fichiers finaux

Une fois que les données sont extraites et que les calculs sont faits, l'application génère automatiquement trois types de fichiers. Voilà comment ça fonctionne pour chacun.

3.3.1. Le fichier Excel d'audit (openpyxl)

Avec la bibliothèque openpyxl, le script crée un classeur Excel formaté avec les tableaux de projection, l'historique des calculs et des graphiques d'évolution (comparaison avec l'inflation ou le Livret A par exemple). La mise en forme respecte les couleurs et la typographie de la marque Le Conservateur. Ce fichier sert de support de travail interne et de trace de l'audit.

3.3.2. Le rapport PDF (reportlab)

Avec reportlab, l'application génère un rapport PDF propre, destiné à être remis au client ou envoyé par mail. Le code gère la mise en page, l'alignement des tableaux, le formatage des montants en euros, et l'intégration des résultats de projection. Le document est immuable et professionnel.

3.3.3. Le PowerPoint de présentation (python-pptx)

C'est probablement le livrable qui a le plus impressionné l'équipe. Avec python-pptx, l'application génère automatiquement un diaporama complet, avec des tableaux remplis,

des indicateurs clés mis en valeur et une mise en forme graphique soignée. Avant, ce fichier demandait plusieurs heures de travail. Maintenant, il est disponible en appuyant sur un seul bouton.

4. La refonte du CRM Excel (Mission 3)

Cette troisième mission n'était pas prévue au départ. Elle s'est présentée en cours de stage, quand l'une des collaboratrices de l'agence m'a demandé de l'aide sur un outil qu'elle avait construit elle-même.

4.1. Le contexte : un outil fait maison qui ne tenait plus

Madame Chatendeau, l'une des collaboratrices indépendantes, avait développé par ses propres moyens un mini-CRM (outil de gestion de la relation client) sous Excel. L'idée de départ était bonne : elle avait besoin de suivre ses prospects, ses clients actifs, ses rendez-vous et ses relances. Comme aucun outil du siège ne répondait à ce besoin précis, elle avait bricolé quelque chose sur Excel.

Sauf qu'avec le temps, le fichier était devenu de plus en plus complexe, avec des formules imbriquées les unes dans les autres, des références croisées partout, et des données qui ne se synchronisaient plus correctement. Les erreurs s'accumulaient. Au lieu d'aider, l'outil était devenu une source de stress.

J'ai organisé une session de travail en visio pour comprendre exactement ce qui ne fonctionnait pas. On a identifié trois problèmes principaux :

- Les nouvelles entrées (nouveaux prospects) ne s'actualisaient plus dans les menus déroulants des autres feuilles.
- Il n'y avait pas de tableau de bord synthétique avec des indicateurs par mois.
- Retrouver l'historique des échanges avec un client spécifique prenait beaucoup de temps et de manipulations.

4.2. Centralisation de la base de données

Le problème de fond, c'était que toutes les données étaient mélangées dans un seul onglet « plat ». Comme une grande liste sans structure claire.

J'ai appliqué les principes des bases de données relationnelles à Excel : j'ai séparé les données en plusieurs tables distinctes (une table Clients, une table Prospects, une table Journal des événements). En utilisant les « Tableaux Structurés » d'Excel et en définissant des identifiants uniques pour chaque contact, j'ai créé des liens entre ces

tables. Quand on modifie une information dans la table principale, elle se met à jour automatiquement partout ailleurs dans le fichier.

4.3. Développement des macros VBA et de l'interface

4.3.1. Un tableau de bord interactif

Pour créer le tableau de bord, j'ai utilisé VBA (Visual Basic for Applications), le langage de programmation intégré à Excel. J'ai automatisé la mise à jour des tableaux croisés dynamiques et ajouté des « Segments » : des boutons interactifs qui permettent de filtrer les données par mois d'un simple clic. En appuyant sur « Mars », par exemple, tous les graphiques et indicateurs se mettent à jour instantanément pour n'afficher que les données de ce mois.

4.3.2. La macro de recherche par double-clic

C'est la fonctionnalité dont Laura était la plus contente. J'ai programmé une macro VBA déclenchée par un double-clic : quand on double-clique sur le nom d'un client dans n'importe quelle liste du fichier, l'algorithme cherche automatiquement dans toute la base de données tous les événements liés à ce client, et affiche son historique complet sur une fiche dédiée.

Ce qui prenait plusieurs minutes de recherche manuelle se fait maintenant en une fraction de seconde.

4.4. Gestion de la confidentialité et RGPD

Le fichier contient des données personnelles (noms, coordonnées, situations financières...). J'ai donc pris soin de mettre en place des mesures simples mais efficaces pour rester dans le cadre du RGPD : accès par mot de passe à certaines feuilles sensibles, séparation claire des données clients et prospects pour limiter les accès croisés, et documentation des données stockées pour faciliter d'éventuelles demandes de suppression.

5. Bilan : résultats, retours et compétences acquises

Une fois les outils développés et mis en place, il est temps de regarder ce que tout ça a vraiment apporté. Est-ce que ça a fonctionné ? Est-ce que les gens s'en servent vraiment ? Et qu'est-ce que j'en retire personnellement ?

5.1. Ce que ça a changé concrètement

5.1.1. Le gain de temps

Le résultat le plus visible, c'est évidemment le temps. Avant, préparer un dossier client prenait entre 2h30 et 3 heures. Avec l'application Python, le même traitement prend moins d'une minute. Le pipeline complet (lecture des PDF, calculs, génération des trois fichiers) s'exécute en quelques dizaines de secondes.

Multiplié par sept conseillers et par le nombre de dossiers traités chaque mois, c'est des centaines d'heures récupérées. Du temps que l'équipe peut désormais investir dans ce qui compte vraiment : voir des clients, développer de nouvelles relations, affiner les stratégies patrimoniales.

5.1.2. La fiabilité des données

L'autre apport majeur, moins visible mais tout aussi important, c'est la fiabilité. Toutes les erreurs liées à la recopie manuelle ont disparu. Les formules de calcul des Tontines et de la prévoyance sont intégrées directement dans le script Python, ce qui garantit que le résultat est juste à chaque fois. Plus de risque de se tromper sur une décimale ou d'utiliser un mauvais coefficient.

Pour les conseillers, c'est une vraie tranquillité d'esprit lors des rendez-vous. Ils savent que les chiffres qu'ils présentent sont exacts. Et pour l'agence, c'est une protection contre les risques légaux liés à une projection erronée.

5.2. Mise en place, manuel et retours de l'équipe

Un outil bien conçu mais mal adopté, ça ne sert à rien. J'ai donc accordé beaucoup d'attention à la phase de déploiement et d'accompagnement.

L'interface de l'application a été volontairement conçue pour être la plus simple possible : un glisser-déposer, un bouton, c'est tout. Pas besoin d'une formation technique pour s'en servir. Les retours de l'équipe ont été immédiatement positifs.

En fin de stage, j'ai rédigé plusieurs documents pour que l'outil continue de fonctionner après mon départ :

- Un manuel d'utilisation simple pour les conseillers, expliquant comment préparer les fichiers sources et utiliser l'application.
- Un guide pour la création des matrices CSV avec l'aide de l'IA, incluant le prompt standardisé à utiliser.
- Une documentation technique pour la gestion du conteneur Docker, à destination de la direction.

L'objectif était que l'agence ne soit pas dépendante de moi pour faire fonctionner ou maintenir les outils.

5.3. Ce que j'ai appris

5.3.1. Les compétences techniques

Sur le plan technique, ce stage m'a permis de mettre en pratique des choses que j'avais apprises en cours, mais aussi d'en découvrir de nouvelles.

- En ingénierie de la donnée, j'ai conçu un pipeline ETL complet, en gérant l'extraction depuis des formats complexes comme les PDF, et en intégrant des outils d'IA (LLM) dans un flux de traitement automatisé.
- En développement logiciel, j'ai traduit des règles de calcul financières complexes (la Tontine, les grilles de prévoyance) en fonctions Python fiables et maintenables.
- En DevOps, j'ai appris à dockeriser une application et à penser le déploiement multi-sites dès la conception.
- En retro-ingénierie, j'ai diagnostiqué et reconstruit un outil CRM existant sous Excel, en appliquant des principes de modélisation relationnelle et de programmation événementielle en VBA.

5.3.2. Les compétences humaines et professionnelles

Au-delà du technique, ce stage m'a aussi beaucoup appris sur le fait de travailler en entreprise.

- Travailler avec des utilisateurs non techniques m'a appris à écouter, à reformuler, à expliquer simplement. Un développeur qui ne sait pas communiquer avec ses utilisateurs finaux, c'est un développeur qui construit des outils que personne n'utilise.
- Avoir géré seul un projet de cette envergure m'a appris à prioriser, à avancer même quand on est bloqué sur un problème, et à savoir abandonner une approche qui ne fonctionne pas (comme l'OCR) pour en essayer une autre.
- La relation avec Monsieur Besson, dans un mode très agile, m'a donné le goût du travail structuré en itérations courtes avec des retours réguliers.

5.4. Ma vision professionnelle

Ce stage a confirmé ce que je pressentais : je veux travailler dans le développement logiciel et l'ingénierie de la donnée. Voir l'impact concret de son code sur le quotidien d'une équipe, c'est une vraie satisfaction.

Les aspects qui m'ont le plus passionné sont le développement back-end, la conception de pipelines de données, et l'intégration d'intelligence artificielle dans des applications concrètes. Je veux continuer dans cette direction, en visant des postes d'ingénieur logiciel ou de data engineer.

Conclusion générale

Ce stage au sein de la SARL Renaud Besson a été une expérience vraiment formatrice. J'ai eu la chance de travailler sur un vrai projet, avec de vraies contraintes, des vrais utilisateurs et de vrais enjeux.

Au départ, l'agence passait des heures à préparer chaque dossier client à la main, avec tous les risques que ça implique. En repartant, j'ai laissé une application web qui fait le même travail en moins d'une minute, un CRM Excel refait de zéro qui fonctionne vraiment, et une documentation complète pour que tout ça continue de tourner.

Sur le plan technique, j'ai pu mettre en œuvre des compétences variées : Python, Flask, Docker, parsing de PDF, intégration d'IA générative, génération automatique de documents, VBA, modélisation de données. Mais ce qui m'a peut-être le plus apporté, c'est la compréhension que la technique n'est qu'un moyen. Ce qui compte, c'est l'impact sur les personnes qui utilisent ce qu'on construit.

Prendre le temps de comprendre un métier, de parler aux utilisateurs, de faire des ajustements en fonction de leurs retours : c'est ça qui fait la différence entre un outil qu'on utilise au quotidien et un outil qu'on range dans un tiroir après deux semaines.

Je repars de cette expérience avec une conviction renforcée sur la direction que je veux donner à ma carrière : l'ingénierie logicielle et l'ingénierie de la donnée, au service de problèmes concrets. Et avec l'envie de continuer à construire des choses utiles.

Résumé

Version française

Ce rapport présente le travail réalisé lors d'un stage de Data Analyst et Développeur BI au sein de la SARL Renaud Besson, agence régionale affiliée au groupe Le Conservateur, spécialisé dans l'ingénierie patrimoniale haut de gamme.

Le constat de départ était simple : préparer un dossier client (extraction de données PDF, recopie depuis la plateforme Qlik Sense, calculs Excel, mise en forme PowerPoint) prenait entre 2h30 et 3 heures par dossier, avec des risques d'erreurs importants.

En réponse à ce problème, j'ai développé une application web automatisée en Python/Flask, déployée via Docker. Elle extrait automatiquement les données des contrats PDF grâce à des algorithmes de parsing par expressions régulières, et intègre une approche IA générative pour pallier l'absence d'export natif de la plateforme Qlik Sense. L'application génère ensuite en moins d'une minute les trois livrables finaux : un audit Excel, un rapport PDF et un diaporama PowerPoint, en intégrant les calculs actuariels complexes de la Tontine.

En parallèle, j'ai diagnostiqué et reconstruit un outil CRM sous Excel/VBA pour une collaboratrice : modélisation relationnelle des données, tableau de bord interactif avec segments, et moteur de recherche par double-clic.

Le résultat principal est une réduction du temps de traitement de 3 heures à moins d'une minute, avec un taux d'erreur réduit à zéro. Ce stage a confirmé mon orientation professionnelle vers le développement logiciel et l'ingénierie de la donnée.

Mots-clés : Python, Flask, Docker, ETL, PDF parsing, IA générative, Excel/VBA, automatisation, gestion de patrimoine, Le Conservateur.

English version

This report presents the work carried out during a Data Analyst & BI Developer internship at SARL Renaud Besson, a regional agency affiliated with the Le Conservateur group, specialising in high-end wealth management.

The starting point was clear: preparing a client file — extracting data from PDF documents, manually copying figures from the Qlik Sense platform, running calculations in Excel, and formatting a PowerPoint presentation — took between 2.5 and 3 hours per file, with significant risk of human error.

To address this, I developed a Python/Flask web application deployed via Docker. It automatically extracts data from PDF contracts using regular expression-based parsing algorithms, and incorporates a generative AI approach to work around the absence of a native data export from the Qlik Sense platform. The application then generates — in under one minute — the three final deliverables: an Excel audit file, a PDF report, and a PowerPoint presentation, including the complex actuarial calculations for the Tontine product.

In parallel, I diagnosed and rebuilt a CRM tool in Excel/VBA for a team member: relational data modelling, an interactive dashboard with slicers, and a double-click-triggered search engine.

The main outcome was a reduction in processing time from 3 hours to under one minute, with an error rate brought down to zero. This internship confirmed my professional direction towards software engineering and data engineering.

Keywords: Python, Flask, Docker, ETL, PDF parsing, generative AI, Excel/VBA, automation, wealth management, Le Conservateur.

Annexes

Les annexes suivantes complètent ce rapport et sont disponibles sur demande ou jointes à ce document :

Cette capture d'écran présente une route Flask en Python (/export-pdf) qui reçoit une liste de membres en JSON, génère un document PDF global (Conclusion) à partir de ces données, puis le renvoie directement en téléchargement au client.

```
@app.route('/export-pdf', methods=['POST'])
def export_pdf():
    data = request.get_json()
    membres = data.get("membres", [])
    if not membres:
        return jsonify({"error": "Aucune donnée de membre reçue"}), 400
    try:
        pdf_buffer = generer_pdf_conclusion(membres)
        nom_fichier = "Famille" if len(membres) > 1 else membres[0].get("nom", "Client").replace(" ", "_")
        return send_file(
            pdf_buffer,
            mimetype="application/pdf",
            as_attachment=True,
            download_name=f"Conclusion_{nom_fichier}.pdf"
        )
    except Exception as e:
        logger.exception("Erreur lors de la génération du PDF global conclusion")
        return jsonify({"error": str(e)}), 500
```

Cette capture d'écran présente la page d'accueil de l'espace de travail de l'application web « Le Conservateur ». Elle met en avant les trois fonctionnalités principales accessibles à l'utilisateur : la gestion des tontines, la création de nouveaux contrats et le suivi des frais kilométriques.



Cette capture d'écran présente l'interface de l'application Container Manager sur un NAS Synology, affichant le conteneur Docker nommé site-conservateur-1 en cours d'exécution. Elle détaille les paramètres techniques essentiels de ce conteneur, tels que la configuration des ports réseau et les volumes de stockage associés.

Container Manager

Vue d'ensemble

Projet

Conteneur

Image

Registre

Réseau

Journal

site-conservateur-1

(En cours d'exécution dans le projet site_conservateur)

Up for 2 days

Arrêt

Dupliquer

Action

Général

Statistiques

Journal

Général

Nom du conteneur	site-conservateur-1
ID	e3bb6656a300501435ba59bdca3a3177d9f358e960cd88333c5d951d0a9d09f5
Nom de l'image	site_conservateur-app-web:latest
Heure de création	11/06/2026 18:26
Heure de début	12/06/2026 18:04 (Up for 2 days)
Projet	site_conservateur
Priorité des processeurs	Moyen
Limite de la mémoire	Sans limite
Activer le redémarrag...	Non
Portails Web	
Port	
5000/tcp	5050
Volume	
	/volume1/docker/app-conservateur:/app:rw
	/volume1/web/frais_kilo:/app/web:rw